

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
JNANA SANGAMA, BELAGAVI - 590018, KARNATAKA



Lab Manual

GENERATIVE AI /
6th Semester (2022 Scheme)
B.E in CSE(AIML)

Prepared by

Dr. Nandeewar S B , Mr. Yasar Beg

Compiled by

Dr. Nandeewar S B

Head of the Department, Dept. of CSE(AIML)



Department of

Computer Science & Engineering
(Artificial Intelligence and Machine Learning)



AMC ENGINEERING COLLEGE



Bannerghatta Road, Bangalore-560083

Affiliated to VTU Belagavi, Approved by AICTE, New Delhi

Course Objective:

Equip learners with practical skills in word embeddings, dimensionality reduction, semantic analysis, and building domain-specific applications using modern NLP tools like Hugging Face, LangChain, and Cohere.

Week-by-Week Syllabus:

Week 1: Introduction to Word Embeddings

- What are word embeddings?
 - Types: Word2Vec, GloVe, FastText
 - Loading pre-trained word vectors using Gensim
 - **Hands-on:** Load and explore word vectors, basic arithmetic operations (king - man + woman = ?)
-

Week 2: Vector Arithmetic & Visualization

- Word similarity and relationships via vector arithmetic
 - Dimensionality reduction techniques (PCA, t-SNE)
 - **Hands-on Project:** Choose 10 domain-specific words (e.g., sports, tech), visualize using t-SNE or PCA, analyze clusters
-

Week 3: Semantic Search & Enrichment

- Generate semantically similar words
 - Contextual enhancement using similar words
 - **Mini Project:** Build a program that:
 - Takes a word input
 - Returns 5 semantically similar words
 - Uses these to enrich prompts for a generative model
 - Compare outputs with/without enrichment
-

Week 4: Creative Writing with Embeddings

- Creative sentence generation with vector spaces
 - **Mini Project:** Input a seed word, generate related words, and create a story or paragraph using them
-

Week 5: Training Custom Word2Vec Models

- Preprocessing domain-specific corpus (e.g., legal/medical)

- Train Word2Vec on custom dataset
 - Compare with pre-trained embeddings
 - **Hands-on:** Build and analyze a legal/medical Word2Vec model
-

Week 6: Sentiment Analysis Using Hugging Face

- Introduction to Hugging Face Transformers
 - Load and use sentiment analysis pipeline
 - Real-world examples: customer reviews, tweets, etc.
 - **Assignment:** Analyze and classify sentiment for a dataset of sentences
-

Week 7: Summarization with Hugging Face

- Use `pipeline('summarization')` for long texts
 - Parameters and fine-tuning for better summaries
 - **Mini Project:** Summarize Wikipedia pages or long news articles
-

Week 8: LangChain + Cohere for Advanced Prompting

- Install LangChain, Cohere, langchain-community
 - Get Cohere API key and integrate it
 - Load document from Google Drive
 - Build a structured prompt template
 - **Hands-on:** Summarize and display output in a formatted style
-

Week 9: Institution Info Extraction via Wikipedia

- Use Pydantic for schema definition
 - Fetch Wikipedia data using LangChain
 - Define output structure (founder, founding year, branches, employees, 4-line summary)
 - **Mini Project:** User inputs institution name → Chain returns structured output
-

Week 10: Indian Penal Code (IPC) Chatbot Project

- Download IPC PDF and preprocess
 - Use vector store + LangChain to enable Q&A
 - Build chat interface
 - Enable follow-up questions, context retention
 - **Final Project:** Fully interactive chatbot that answers queries about Indian Penal Code
-

Table of Content

Generative AI		Semester	6
Course Code	BAIL657C	CIE Marks	50
Teaching Hours/Week(L:T:P:S)	0:0:1:0	SEE Marks	50
Credits	01	Exam Hours	100
Examination type(SEE)	Practical		
Course objectives: • Understand the principles and concepts behind generative AI models • Explain the knowledge gained to implement generative models using Prompt design frameworks. • Apply various Generative AI applications for increasing productivity. • Develop Large Language Model-based Apps.			
Sl.NO	Experiments		
1.	Explore pre-trained word vectors. Explore word relationships using vector arithmetic. Perform arithmetic operations and analyze results.		
2.	Use dimensionality reduction (e.g., PCA or t-SNE) to visualize word embeddings for Q1.Select 10 words from a specific domain (e.g., sports, technology) and visualize their embeddings. Analyze clusters and relationships. Generate contextually rich outputs using embeddings. Write a program to generate 5 semantically similar words for a given input.		
3.	Train a custom Word2Vec model on a small dataset. Train embeddings on a domain-specific corpus (e.g., legal, medical) and analyze how embeddings capture domain-specific semantics.		
4.	Use word embeddings to improve prompts for Generative AI model.Retrieve similar words using word embeddings. Use the similar words to enrich a GenAI prompt. Use the AI model to generate responses for the original and enriched prompts. Compare the outputs in terms of detail and relevance.		
5.	Use word embeddings to create meaningful sentences for creative tasks. Retrieve similar words for a seed word. Create a sentence or story using these words as a starting point. Write a program that:Takes a seed word.Generates similar words. Constructs a short paragraph using these words.		
6.	Use a pre-trained Hugging Face model to analyze sentiment in text. Assume a real-world application, Load the sentiment analysis pipeline. Analyze the sentiment by giving sentences to input.		
7.	Summarize long texts using a pre-trained summarization model using Hugging face model. Load the summarization pipeline. Take a passage as input and obtain the summarized text.		
8.	Install langchain, cohere (forkey), langchain-community. Get the api key(By logging into Cohere and obtaining the cohere key). Load a text document from your google drive . Create a prompt templateto display the output in a particular manner.		
9.	Take the Institution name as input. Use Pydantic to define the schema for the desired output and create a custom output parser. Invoke the Chain and Fetch Results. Extract the below Institution related details from Wikipedia: The founder of the Institution. When it was founded. The current branches in the institution. How many employees are working in it. A brief4-line summary of the institution.		
10	Build a chatbot for the Indian Penal Code. We'll start by downloading the official Indian Penal Code document, and then we'll create a chatbot that can interact with it. Users will be able to ask questions about the Indian Penal Code and have a conversation with it.		

Course outcomes (Course Skill Set):

At the end of the course the student will be able to:

- Develop the ability to explore and analyze word embeddings, perform vector arithmetic to investigate word relationships, visualize embeddings using dimensionality reduction techniques
- Apply prompt engineering skills to real-world scenarios, such as information retrieval, text generation.
- Utilize pre-trained Hugging Face models for real-world applications, including sentiment analysis and text summarization.
- Apply different architectures used in large language models, such as transformers, and understand their Advantages and limitations.

Assessment Details (both CIE and SEE)

The weight age of Continuous Internal Evaluation (CIE) is 50% and for Semester End Exam (SEE) is 50%. The minimum passing mark for the CIE is 40% of the maximum marks (20marks out of 50) and for the SEE minimum passing mark is 35% of the maximum marks(18out of 50 marks).A student shall be deemed to have satisfied the academic requirements and earned the credits allotted to each subject/course if the student secures a minimum of 40%(40marks out of 100)in the sum total of the CIE(Continuous Internal Evaluation) and SEE (Semester End Examination)taken together

Continuous Internal Evaluation(CIE):

CIE marks for the practical course are **50 Marks**.

The split-up of CIE marks for record/journal and test are in the ratio **60:40**.

- Each experiment is to be evaluated for conduction with an observation sheet and record write-up. Rubrics for the evaluation of the journal/write-up for hardware/software experiments are designed by the faculty who is handling the laboratory session and are made known to students at the beginning of the practical session.
- Record should contain all the specified experiments in the syllabus and each experiment write-up will be evaluated for 10 marks.
- Total marks scored by the students are scaled down to **30marks** (60% of maximum marks).
- Weight age to be given for neatness and submission of record/write-up ontime.
- Department shall conduct a test of 100 marks after the completion of all the experiments listed in the syllabus.
- In a test, test write-up, conduction of experiment, acceptable result, and procedural knowledge will carry a weight age of 60% and the rest 40% for viva-voce.
- The suitable rubrics can be designed to evaluate each student's performance and learning ability.
- The marks scored shall be scaled down to **20 marks** (40%of the maximum marks).

The Sum of scaled-down marks scored in the report write-up /journal and marks of a test is the total CIE marks scored by the student.

Semester End Evaluation(SEE):

- SEE marks for the practical course are 50 Marks.
- SEE shall be conducted jointly by the two examiners of the same institute, examiners are appointed by the Head of the Institute.

- The examination schedule and names of examiners are informed to the university before the conduction of the examination. These practical examinations are to be conducted between the schedule mentioned in the academic calendar of the University.
- All laboratory experiments are to be included for practical examination.
- (Rubrics) Break up of marks and the instructions printed on the cover page of the answer script to be strictly adhered to by the examiners. **OR** based on the course requirement evaluation rubrics shall be decided jointly by examiners.
- Students can pick one question (experiment) from the questions lot prepared by the examiners jointly.
- Evaluation of test write-up /conduction procedure and result/viva will be conducted jointly by examiners.

General rubrics suggested for SEE are mentioned here , writeup-20%, Conduction procedure and result in-60%, Viva-voce 20% of maximum marks. SEE for practical shall be evaluated for 100 marks and scored marks shall be scaled down to 50 marks (however,based on course type, rubrics shall be decided by the examiners)

Change of experiment is allowed only once and 15% of Marks allotted to the procedure part are to be made zero.

The minimum duration of SEE is 02 hours

Tools & Libraries Used:

- Python
- Gensim, Numpy, Scikit-learn
- Hugging Face Transformers
- LangChain
- Cohere API
- Pydantic
- t-SNE, PCA (Matplotlib/Plotly for visualization)

General Questions & Answers

1. What is Generative AI?

Generative AI, short for Generative Artificial Intelligence, is a subset of artificial intelligence (AI) that focuses on enabling machines to produce content or data that resembles human-generated information. It's a technology that's gaining immense popularity in various fields, from natural language processing to creative content generation.

Generative AI operates on a principle of learning patterns from existing data and using that knowledge to create new content. It relies on deep learning techniques, particularly neural networks, to accomplish this task. These neural networks are trained on large datasets, allowing them to generate text, images, music, and more.

2. How does Generative AI work?

Generative AI works through the use of neural networks, specifically [Recurrent Neural Networks](#) (RNNs) and more recently, Transformers. Here's a simplified breakdown of how it functions:

- **Data Collection:** To begin, a substantial amount of data related to the specific task is gathered. For instance, if you want to generate text, the model needs a massive text corpus to learn from.
- **Training:** The neural network is then trained on this data. During training, the model learns the underlying patterns, structures, and relationships within the data. It learns to predict the next word, character, or element in a sequence.
- **Generation:** Once trained, the model can generate content by taking a seed input and predicting the subsequent elements. For instance, if you give it the start of a sentence, it can complete the sentence in a coherent and contextually relevant manner.
- **Fine-Tuning:** Generative AI models can be further fine-tuned for specific tasks or domains to improve the quality of generated content.

3. What are the top applications of Generative AI?

Generative AI has a wide range of applications across different industries:

- **Natural Language Processing (NLP):** It's used for text generation, language translation, and chatbots that can engage in human-like conversations.
- **Content Generation:** Generative AI can create articles, stories, and even poetry. It's used by content creators to assist in writing.
- **Image and Video Generation:** It can generate realistic images and videos, which are valuable in fields like entertainment and design.
- **Data Augmentation:** In data science, it's used to create synthetic data for training machine learning models.
- **Healthcare:** Generative AI helps in generating medical reports, simulating disease progression, and drug discovery.

4. Can you explain the difference between Generative AI and Discriminative AI?

Aspect	Generative AI	Discriminative AI
Objective	Generates new data based on learned patterns.	Classifies input data into predefined classes.
Function	Content creation, data generation.	Classification, discrimination.
Use Cases	Text generation, image synthesis, creativity.	Spam detection, sentiment analysis, image recognition.
Learning Process	Learns patterns in data for content generation.	Focuses on learning boundaries between classes.
Examples	Chatbots, text generators, art creation.	Spam filters, image classifiers, sentiment analysis models.

5. What are some popular Generative AI models?

Generative AI models have revolutionized the field of artificial intelligence, offering remarkable capabilities in generating content, from text to images and beyond. In this section, we'll explore some of the most popular and influential Generative AI models that have left a significant mark on the industry.

1. **a. GPT-4 (Generative Pre-trained Transformer 4):** GPT-4, developed by OpenAI, is a standout among Generative AI models. With billions of parameters, it has demonstrated remarkable text generation abilities. [GPT-4](#) can answer questions, write essays, generate code, and even create conversational agents that engage users in natural language.
2. **BERT (Bidirectional Encoder Representations from Transformers):** Although primarily known for its prowess in natural language understanding, [BERT](#) also exhibits generative capabilities. It excels in tasks like text completion and summarization, making it a valuable tool in various applications, including search engines and chatbots.
3. **DALL-E:** If you're interested in generative art, [DALL-E](#) is a model to watch. Developed by OpenAI, this model can generate images from textual descriptions. It takes creativity to new heights by creating visuals based on written prompts, showing the potential of Generative AI in the visual arts.
4. **StyleGAN2:** When it comes to generating realistic images, StyleGAN2 is a name that stands out. It can create high-quality, diverse images that are virtually indistinguishable from real photographs. StyleGAN2 has applications in gaming, design, and even fashion.

5. **VQ-VAE-2 (Vector Quantized Variational Autoencoder 2):** This model combines elements of generative and variational autoencoders to generate high-quality, high-resolution images. It has made significant strides in image compression and generation.

6. How is Generative Adversarial Networks (GANs) used in AI?

[Generative Adversarial Networks](#), or GANs, have emerged as a groundbreaking concept in the realm of Generative AI. These networks consist of two primary components: a generator and a discriminator, which work in tandem to create and evaluate content. Here's how GANs are used in AI:

Application	Description
Image Generation	Create high-quality images, artworks, and more.
Data Augmentation	Generate additional data to enhance training datasets.
Style Transfer	Transform the style of images, e.g., artist-inspired styles.
Super-Resolution	Enhance image resolution for clarity and detail.
Anomaly Detection	Identify deviations from the normal data distribution.
Text-to-Image Generation	Generate images from textual descriptions.

Top Applications of GANs:

- **Image Generation:** GANs are widely used to create high-quality images, such as faces of non-existent individuals, realistic artworks, and more. This is particularly valuable in creative industries and design.
- **Data Augmentation:** GANs can generate additional data to augment training datasets. This is crucial in scenarios where obtaining large amounts of real data is challenging.
- **Style Transfer:** GANs can transform the style of images, such as converting a photograph into the style of a famous artist. This has applications in art and design.
- **Super-Resolution:** GANs can enhance the resolution of images, making them sharper and clearer. This is beneficial in fields like medical imaging and photography.
- **Anomaly Detection:** GANs can be used to detect anomalies in data by learning the normal distribution of data and flagging deviations from it.
- **Text-to-Image Generation:** GANs can generate images from textual descriptions, opening up possibilities in e-commerce and visual storytelling.

7. What are the limitations of Generative AI?

While Generative AI has made remarkable strides, it's essential to acknowledge its limitations and challenges. Understanding these limitations is crucial for responsible and effective use. Here are some key constraints of Generative AI:

1. **Data Dependency:** Generative AI models, including GANs, require vast amounts of data for training. Without sufficient data, the quality of generated content may suffer, and the model might produce unrealistic or biased results.
2. **Ethical Concerns:** Generative AI can inadvertently perpetuate biases present in the training data. This raises ethical concerns, particularly when it comes to generating content related to sensitive topics, such as race, gender, or religion.
3. **Lack of Control:** Generative AI can be unpredictable. Controlling the output to meet specific criteria, especially in creative tasks, can be challenging. This lack of control can limit its practicality in some applications.
4. **Resource Intensive:** Training and running advanced Generative AI models demand substantial computational resources, making them inaccessible to smaller organizations or individuals with limited computing power.
5. **Overfitting:** Generative models may memorize the training data instead of learning its underlying patterns. This can result in content that lacks diversity and creativity.
6. **Security Risks:** There is the potential for malicious use of Generative AI, such as generating deepfake videos for deceptive purposes or creating fake content to spread misinformation.
7. **Intellectual Property Concerns:** When Generative AI is used to create content, determining ownership and copyright becomes complex. This raises legal questions about intellectual property rights.
8. **Validation Challenges:** It can be difficult to validate the authenticity of content generated by Generative AI, which can be problematic in contexts where trust and reliability are paramount.

8. What are the ethical concerns surrounding Generative AI?

Generative AI, with its ability to create content autonomously, brings forth a host of ethical considerations. As this technology becomes more powerful, it's crucial to address these concerns to ensure responsible and ethical use. Here are some of the ethical concerns surrounding Generative AI:

1. **Bias and Fairness:** Generative AI models can inadvertently perpetuate biases present in their training data. This can lead to the generation of content that reflects and reinforces societal biases related to race, gender, and other sensitive attributes.
2. **Privacy:** Generative AI can be used to create deepfake content, including fabricated images and videos that can infringe upon an individual's privacy and reputation.
3. **Misinformation:** The ease with which Generative AI can generate realistic-looking text and media raises concerns about its potential for spreading misinformation and fake news.

4. **Identity Theft:** Generative AI can create forged identities, making it a potential tool for identity theft and fraud.
5. **Deceptive Content:** Malicious actors can use Generative AI to create deceptive content, such as fake reviews, emails, or social media posts, with the intent to deceive or defraud.
6. **Legal and Copyright Issues:** Determining the legal ownership and copyright of content generated by AI can be complex, leading to legal disputes and challenges.
7. **Psychological Impact:** The use of Generative AI in creating content for entertainment or social interactions may have psychological impacts on individuals who may not always distinguish between AI-generated and human-generated content.
8. **Accountability:** Establishing accountability for content generated by AI is challenging. When harmful content is created, it can be unclear who should be held responsible.

To address these ethical concerns, developers and users of Generative AI must prioritize responsible and ethical practices. This includes rigorous data curation to minimize bias, clear labeling of AI-generated content, and adherence to ethical guidelines and regulations.

9. How can Generative AI be used in art and creativity?

Use Case	Description
Art Generation	AI algorithms create visual art based on input parameters.
Music Composition	AI generates music, offering fresh inspiration to musicians.
Writing Assistance	AI assists writers with ideas, plot twists, and even stories.
Design Optimization	AI optimizes layouts, colors, and styles in design fields.
Art Restoration	AI reconstructs damaged artworks, preserving cultural heritage.
Style Transfer	AI applies artistic styles to photos or images, creating unique visuals.
Virtual Worlds	AI powers immersive virtual worlds, enhancing gaming and entertainment.
Fashion Design	AI generates clothing designs, predicting trends in fashion.

10. What are the challenges in training Generative AI models?

Training Generative AI models presents several challenges:

1. **Data Quality:** High-quality training data is essential. Noisy or biased data can lead to flawed outputs.
2. **Computational Resources:** Training large models demands substantial computational power and time.
3. **Mode Collapse:** GANs may suffer from mode collapse, where they generate limited varieties of outputs.
4. **Ethical Considerations:** AI-generated content can raise ethical issues, including misinformation and deepfakes.
5. **Evaluation Metrics:** Measuring the quality of generated content is subjective and requires robust evaluation metrics.

11. What are the key components of a GAN architecture?

A Generative Adversarial Network (GAN) comprises two main components:

1. **Generator:** This component creates synthetic data. It takes random noise as input and transforms it into data that resembles the training dataset.
2. **Discriminator:** The discriminator's role is to distinguish between real and generated data. It learns to classify data as real or fake.

GANs operate on a feedback loop. The generator aims to produce data that can fool the discriminator, while the discriminator gets better at distinguishing real from fake data. This competition results in the generation of high-quality synthetic content.

12. How does text generation with Generative AI work?

Text generation with Generative AI involves models like GPT (Generative Pre-trained Transformer). Here's how it works:

1. **Pre-training:** Models are initially trained on a massive corpus of text data, learning grammar, context, and language nuances.
2. **Fine-tuning:** After pre-training, models are fine-tuned on specific tasks or datasets, making them domain-specific.
3. **Autoregressive Generation:** GPT generates text autoregressively, predicting the next word based on context. It's conditioned on input text.
4. **Sampling Strategies:** Techniques like beam search or temperature-based sampling control the creativity and diversity of generated text.

13. Can Generative AI create realistic images and videos?

Generative AI, including models like Generative Adversarial Networks (GANs) and [Variational Autoencoders](#) (VAEs), has made remarkable strides in creating realistic images and videos. These technologies are at the forefront of modern artificial intelligence, bridging the gap between creativity and technology.

Generative AI accomplishes this feat by learning from vast datasets of real-world images and videos. It then employs a two-step process to generate new content. Here's how it works:

- **Generator Network:** The generator network takes random noise as input and attempts to produce data that resembles real images or videos. This network is responsible for the creative aspect, introducing variations and uniqueness into the content.
- **Discriminator Network:** Simultaneously, there's a discriminator network that evaluates the content generated by the generator. Its role is to distinguish between real and generated content. It's like a detective trying to spot fake art from genuine masterpieces.

These two networks engage in a continuous battle. The generator aims to produce content that fools the discriminator into believing it's real, while the discriminator becomes increasingly skilled at telling the difference. This back-and-forth training process eventually results in the generator creating highly realistic images and videos.

14. How does StyleGAN work, and what are its applications?

StyleGAN is a cutting-edge Generative Adversarial Network (GAN) variant renowned for its ability to generate high-resolution, realistic images with an unprecedented level of control and customization.

At its core, StyleGAN operates by separating the generation process into two crucial components: the style and the structure.

- **Style Mapping:** StyleGAN starts by mapping a latent vector (essentially a set of random numbers) into a style space. This style space controls various high-level features of the generated image, such as the pose, facial expression, and overall aesthetics. This separation of style from structure allows for precise control over these attributes.
- **Synthesis Network:** The second part involves a synthesis network that generates the image structure based on the learned style. This network uses convolutional layers to create the image pixel by pixel, guided by the style information. This separation of style and structure allows for incredible flexibility and customization.

Applications:

Applications of StyleGAN	Description
Art and Fashion	Create customizable art pieces and fashion designs with unique aesthetics.
Facial Generation	Generate realistic faces for video games, digital characters, and movie special effects.
Data Augmentation	Diversify datasets for machine learning, improving model training and performance.

Content Creation	Produce unique visuals, logos, and branding materials for various creative purposes.
Realistic Image Editing	Edit images while maintaining authenticity, enabling advanced image manipulation.

15. Are there any Generative AI models used in natural language processing (NLP)?

Generative AI models have made significant strides in the field of [Natural Language Processing \(NLP\)](#), revolutionizing the way machines understand and generate human language. One of the most prominent examples is the use of Transformers, a class of generative models that has reshaped NLP.

Transformers, which includes models like GPT-4 (Generative Pre-trained Transformer 4) and BERT (Bidirectional Encoder Representations from Transformers), have demonstrated remarkable capabilities in understanding and generating natural language text.

Here's how they work:

- **Attention Mechanism:** Transformers utilize an attention mechanism that allows them to weigh the importance of each word or token in a sentence concerning others. This mechanism helps the model capture context effectively.
- **Pre-training:** These models are pre-trained on a vast corpora of text data. During this phase, they learn grammar, facts, and even some reasoning abilities from the text. For example, they can predict the next word in a sentence or mask a word and predict it based on the surrounding context.
- **Fine-tuning:** After pre-training, models like GPT-3 or BERT are fine-tuned on specific NLP tasks like language translation, sentiment analysis, or question-answering. This fine-tuning tailors the model to excel in these particular tasks.

16. How can Generative AI be used in healthcare?

Generative AI in Healthcare	Technical Applications
Medical Imaging	Enhancing image quality for diagnosis.
Drug Discovery	Generating molecular structures for new drugs.
Health Data Generation	Synthesizing medical data for ML datasets.
Predictive Modeling	Creating models for disease outbreak prediction.

Natural Language Processing	Generating medical reports and clinical notes.
Personalized Medicine	Tailoring treatment plans based on patient data.
Medical Simulations	Creating realistic training simulations for healthcare professionals.

17. What role does reinforcement learning play in Generative AI?

Reinforcement learning, a pivotal branch of artificial intelligence, plays a substantial role in the realm of Generative AI. At its core, reinforcement learning involves training models to make sequences of decisions by interacting with an environment.

In Generative AI, reinforcement learning is often employed to enhance the generation process.

Here's how it works:

The AI model generates an output, such as an image or text; then, reinforcement learning comes into play by evaluating the quality of that output. If it's subpar, the model adjusts its internal parameters to generate better results.

This iterative process continues, gradually improving the AI's ability to create content. It's particularly beneficial when precision and fine-tuning are essential, as in applications like natural language generation and image synthesis.

18. What is the importance of data in training Generative AI models?

Data is the lifeblood of Generative AI models. The quality and quantity of data used in training have a profound impact on the model's performance. Generative AI models learn from data, seeking patterns and structures within it to generate new content.

For instance, in text generation, a model trained on a diverse and extensive dataset can produce more coherent and contextually relevant text. In image generation, the richness of data influences the model's ability to create high-resolution and visually pleasing images.

Moreover, data diversity is vital. Training data should encompass various styles, contexts, and nuances to enable the AI model to adapt to different scenarios. Without robust data, Generative AI models would lack the foundation needed for creativity and accuracy.

19. Can Generative AI be used for anomaly detection?

Yes, Generative AI can be a powerful tool for anomaly detection. Anomaly detection involves identifying patterns or instances that deviate significantly from the norm within a dataset. Generative AI models, such as autoencoders and GANs (Generative Adversarial Networks), excel in this area.

Autoencoders, for example, are neural networks designed to reconstruct their input data. When trained on normal data, they become adept at reproducing it accurately. However, when presented with anomalies, they struggle to reconstruct them accurately, highlighting deviations.

Similarly, GANs can generate data that mimics the training dataset’s characteristics. Any data that significantly differs from the generated samples is flagged as an anomaly. This application is valuable in various domains, including fraud detection and cybersecurity.

20. What are some examples of Generative AI generating music?

Generative AI Music Tools	Key Features
Meta’s AudioCraft	– Trained on licensed music and sound effects. – Enables quick addition of music and sounds to videos via text prompts.
OpenAI’s MuseNet	– Analyzes style, rhythm, and harmony in music. – Can switch between music genres and incorporate up to 10 instruments.
iZotope’s AI Assistants	– Pioneering AI-assisted music production tool. – Offers valuable insights and assistance in music creation.
Jukebox by OpenAI	– Generates music samples from scratch based on genre, artist, and lyrics.
VEED’s AI Music Generator	– Creates royalty-free, unique soundtracks for videos using Generative AI.

21. How does Generative AI impact content generation on the internet?

Aspect	Description
Efficiency	Rapidly generates large amounts of content
Personalization	Tailors content to individual user preferences
Multilingual Support	Creates content in multiple languages
SEO Optimization	Analyzes keywords for better search engine ranking
Content Variability	Produces diverse content types for wider engagement
Quality Control	Requires human oversight for accuracy and coherence

22. What are some successful real-world applications of Generative AI?

Application	Example
Image Generation	OpenAI's DALL-E generated an image from text descriptions
Conversational AI Apps for Patients	Ada: Symptom assessment and medical guidance in multiple languages
AI for Early Disease Detection	SkinVision: Early detection of skin cancer
AI for Accessibility	Be My Eyes: Converting images to text for the visually impaired
AI for Patient Interactions and Support	Hyro: Enhancing patient engagement and healthcare support
Content Creation	ChatGPT: Generating text content and creative writing

23. How do you evaluate the quality of output from a Generative AI model?

Evaluation Aspect	Description
Human Review	Assess output for coherence, relevance, and accuracy
Diversity Check	Ensure content doesn't become repetitive
Plagiarism Detection	Verify originality and copyright compliance
User Feedback	Gather user input for improvement
Domain-Specific Metrics	Use metrics like BLEU scores for specific domains
Ethical Considerations	Ensure content aligns with ethical guidelines

24. Can Generative AI be used for language translation?

Yes, Generative AI is increasingly used for language translation, and it has significantly improved the accuracy and efficiency of translation services. Here's how it works:

- **Neural Machine Translation (NMT):** Generative AI models, particularly those based on NMT, excel at language translation. They analyze vast amounts of bilingual text data to learn how languages correspond and then generate translations based on this knowledge.
- **Multilingual Capabilities:** These models can handle multiple languages, making them versatile for global communication.
- **Continuous Improvement:** AI translation models continuously learn and adapt to language nuances, ensuring that translations become more accurate over time.
- **Real-time Translation:** AI-powered translation services are integrated into various platforms, allowing for real-time translation of text, speech, and even images.

25. What are the privacy concerns related to Generative AI?

Privacy concerns surrounding Generative AI have become increasingly prominent in recent years. As these powerful AI models, like GPT-4, continue to evolve, several key issues have emerged:

- **Data Privacy:** Generative AI models require vast amounts of data to train effectively. This raises concerns about the privacy of the data used, as it may include sensitive or personal information.
- **Bias and Fairness:** Generative AI models can inadvertently perpetuate biases present in their training data. This can lead to biased or unfair outputs, impacting various applications from content generation to decision-making.
- **Deepfakes and Misinformation:** Generative AI can be used to create highly convincing deepfake videos and text, making it challenging to distinguish between real and fabricated content, thus fueling the spread of misinformation.
- **Security Risks:** Malicious actors can misuse Generative AI to automate phishing attacks, create fake identities, or generate fraudulent content, posing significant security risks.
- **User Privacy:** As AI models generate personalized content, there is a concern about user privacy. How much personal information should be input for customization, and how securely is it stored?

To address these concerns, researchers and developers are actively working on improving transparency, fairness, and privacy-preserving techniques in Generative AI. It's crucial to strike a balance between the power of these models and the potential risks they pose to privacy.

26. How can Generative AI models be fine-tuned for specific tasks?

Steps	Description
-------	-------------

Step 1: Dataset Selection	Choose a relevant, diverse dataset.
Step 2: Architecture Selection	Pick a suitable pre-trained model.
Step 3: Task-Specific Objective	Define a clear task and adapt the model.
Step 4: Hyperparameter Tuning	Adjust parameters for optimal performance.
Step 5: Training Process	Train the model and monitor performance.
Step 6: Regularization Techniques	Apply techniques like dropout and decay.
Step 7: Evaluation	Assess performance using relevant metrics.

27. What are some challenges in making Generative AI models more efficient?

Efficiency is a critical aspect of Generative AI models. Several challenges need to be overcome to make these models more efficient:

- **Computational Resources:** Training and running large AI models demands significant computational power, making them inaccessible for many users.
- **Model Size:** The sheer size of models like GPT-3 poses challenges in terms of memory and storage requirements.
- **Inference Speed:** Real-time applications require models that can generate responses quickly, which can be a challenge for complex Generative AI models.
- **Energy Consumption:** Running large models consumes a substantial amount of energy, which is not environmentally sustainable.
- **Scalability:** Scaling up AI models to handle diverse tasks while maintaining efficiency is a complex task.

28. Can Generative AI be used for generating 3D models?

Yes, Generative AI can be harnessed for 3D model generation. This exciting application has gained traction in recent years. Here's how it works:

- **Data Preparation:** Generative AI models require 3D training data, which can include images, point clouds, or even existing 3D models.
- **Model Architecture:** Specialized architectures like 3D-GANs (Generative Adversarial Networks) or VAEs (Variational Autoencoders) are used for 3D model generation.

- **Training:** The model is trained to generate 3D structures based on the provided data. This can be used for creating 3D objects, scenes, or even medical images.
- **Applications:** 3D Generative AI finds applications in various fields, including gaming, architectural design, medical imaging, and manufacturing, enabling the automated creation of 3D content.

29. How does Generative AI assist in generating new product designs?

Generative AI is revolutionizing the field of product design. It leverages deep learning algorithms to analyze vast datasets of existing designs, user preferences, and market trends. By doing so, it assists designers in generating innovative and unique product concepts. Here's how it works:

Generative AI algorithms, such as Generative Adversarial Networks (GANs) and Variational Autoencoders (VAEs), learn patterns and features from large datasets of product designs.

These algorithms can then generate new design variations based on the learned patterns. This not only accelerates the design process but also opens the door to entirely novel ideas.

Designers can input specific constraints or preferences, and Generative AI will adapt the generated designs accordingly. This level of customization is a game-changer in product development.

Generative AI also aids in rapid prototyping, allowing designers to explore multiple design options quickly.

In summary, Generative AI empowers designers by offering a wealth of design possibilities, streamlining the creative process, and ultimately leading to the creation of more innovative products.

30. Are there any Generative AI models that generate code?

Yes, there are Generative AI models specifically designed for code generation. These models are a boon for developers, as they automate and optimize many aspects of software development. Here's an overview:

- One prominent example is OpenAI's GPT-4, which can generate code snippets for a variety of programming languages.
- Another noteworthy model is OpenAI's Codex, built on GPT-3, which excels at understanding and generating code in natural language. It's like having a coding companion.
- Generative AI models analyze code repositories and documentation to understand coding conventions and best practices. They can then generate code that aligns with these standards.
- These models are not just limited to generating simple code snippets; they can assist in more complex tasks, such as writing entire functions or even suggesting optimized algorithms.
- Developers can save time and reduce errors by leveraging Generative AI models for code generation, making software development more efficient.

31. What is the role of Generative AI in generating realistic game environments?

Generative AI plays a pivotal role in the gaming industry, enhancing the creation of immersive and realistic game environments. Here's how it contributes:

- Generative AI algorithms, particularly procedural content generation (PCG), can generate vast and diverse game worlds. These algorithms use mathematical rules to create terrain, landscapes, and structures, reducing the need for manual design.
- Realistic textures and 3D models can be generated with the help of Generative AI, making game environments visually stunning.
- Dynamic storytelling within games benefits from Generative AI's ability to create branching narratives and adapt to player choices, resulting in a more engaging player experience.
- Generative AI can simulate natural behaviors for in-game characters, making NPCs (non-playable characters) and enemies more lifelike and responsive.

32. How does Generative AI influence storytelling and narrative generation?

Generative AI is reshaping storytelling by providing powerful tools for authors and content creators. Here's how it influences narrative generation:

- Generative AI models can analyze vast amounts of text data to understand storytelling patterns, character development, and plot structures.
- Authors can use Generative AI to brainstorm ideas, generate plot outlines, or even create character dialogues that fit seamlessly within a story.
- These models can adapt their output to match a specific writing style or genre, making them versatile tools for authors with varying creative needs.
- In collaborative writing, Generative AI can suggest plot twists, character arcs, or even entire chapters, fostering creativity and efficiency among writers.
- Generative AI is not here to replace authors but to assist and inspire, making the storytelling process more efficient and imaginative.

33. Can Generative AI be used for data augmentation in machine learning?

Generative AI, a remarkable branch of artificial intelligence, plays a pivotal role in enhancing machine learning models through data augmentation. It's a technique that resonates with both beginners and seasoned professionals.

Data augmentation is the process of increasing the diversity and volume of training data to improve the robustness and accuracy of machine learning models. Generative AI, with its ability to generate synthetic data, has found a crucial application in this domain.

Using Generative Adversarial Networks (GANs) and other generative techniques, data scientists can create realistic data points that closely mimic the distribution of the original dataset. This synthetic data can then be added to the training set, effectively increasing its size and variety.

The benefits are twofold. First, it helps prevent overfitting by providing more examples for the model to learn from. Second, it aids in addressing data scarcity issues, especially in niche domains where collecting extensive data is challenging.

However, it’s essential to ensure that the generated data is of high quality and representative of the real-world scenarios. Rigorous validation and testing are crucial steps in this process to maintain the integrity of the model.

34. What are the future prospects of Generative AI?

Aspect	Future Outlook
Improved Realism	Enhanced realism in generated content.
Industry Applications	Diverse applications across industries.
Ethical Considerations	Addressing ethical concerns around deepfakes.
Healthcare Innovations	Advancements in medical imaging and drug discovery.
Personalized Content	Tailored content generation for users in real-time.
Climate Modeling	Improved climate change simulations and predictions.

35. How is Generative AI used in generative design in architecture and engineering?

Generative AI revolutionizes architecture and engineering by enabling generative design. This approach leverages algorithms to explore countless design possibilities. Here’s how Generative AI is making its mark:

Generative algorithms analyze parameters and constraints provided by architects and engineers to generate designs. This iterative process leads to innovative solutions, optimizing structures for functionality, aesthetics, and sustainability.

36. What are the security implications of Generative AI?

Concern	Description
Deepfakes	Realistic fake videos for potential misuse.
Data Privacy	Privacy risks related to data generation.

Authentication	Challenges to traditional authentication methods.
Identity Theft	Potential for generating convincing fake identities.
Content Manipulation	Altered content may deceive and spread misinformation.
Cyberattacks	AI-generated cyber threats and attacks.

37. How can Generative AI be used in personalized content recommendation?

Generative AI plays a pivotal role in personalized content recommendation systems. By utilizing advanced machine learning algorithms, Generative AI tailors content suggestions based on individual preferences and behavior. Here's how it works:

Generative AI models, such as GPT-4, analyze user data like past browsing history, search queries, and interactions with content. These models then generate recommendations that are highly relevant to each user.

For example, if you're an e-commerce platform, Generative AI can suggest products based on a user's previous purchases and the preferences of similar users. This personalization enhances user engagement and boosts conversion rates.

38. What are the hardware requirements for training large Generative AI models?

Hardware Component	Requirements and Description
Graphics Processing Units	High-performance GPUs or TPUs are essential for processing complex computations during model training. Multiple GPUs in a cluster can significantly speed up the process.
Memory Capacity	Large memory capacity is crucial for storing model parameters, especially in the case of large Generative AI models.
Storage	Fast storage solutions, such as Solid State Drives (SSDs), are used to enable quick data retrieval and storage during training.
Computing Clusters	Distributed computing clusters with multiple GPUs are employed for parallel processing, reducing training time.

Internet Connection

Access to high-speed internet is necessary for downloading and transferring large datasets, as well as for accessing cloud-based resources for training.

39. How does unsupervised learning relate to Generative AI?

Unsupervised learning is at the core of Generative AI. It's a machine learning paradigm where models learn from unlabeled data, finding hidden patterns and structures. Generative AI leverages unsupervised learning to create data or content that resembles human-generated data.

For example, unsupervised learning is used in Generative Adversarial Networks (GANs), a popular Generative AI architecture. GANs consist of a generator and a discriminator network that compete with each other. The generator aims to create realistic data (like images or text), while the discriminator tries to distinguish between real and generated data. This adversarial process drives the generator to produce increasingly authentic content.

40. Can Generative AI be used in drug discovery and molecular design?

Absolutely, Generative AI is a game-changer in drug discovery and molecular design. It expedites the process of identifying potential drug candidates and designing new molecules with specific properties.

Generative AI models can predict molecular properties, generate novel chemical structures, and optimize existing compounds. This accelerates drug development, making it more cost-effective and efficient.

Researchers and pharmaceutical companies are utilizing Generative AI to simulate molecular interactions, screen potential drugs, and discover novel solutions for challenging medical conditions.

Program 1 :

Explore pre-trained word vectors. Explore word relationships using vector arithmetic. Perform arithmetic operations and analyze results.

```
import numpy as np

from gensim.downloader import load

import pprint

model = load('fasttext-wiki-news-subwords-300') # 1 GB model

print("Similar words to 'king': ")

pprint.pprint(model.most_similar('king'))

Similar words to 'king':
[('king-', 0.78380286693573),
 ('boy-king', 0.7704817056655884),
 ('queen', 0.7704246640205383),
 ('prince', 0.7700967192649841),
 ('kings', 0.7668929696083069),
 ('sub-king', 0.7391484379768372),
 ('monarch', 0.736833393573761),
 ('king-making', 0.7288671731948853),
 ('ex-king', 0.7280158996582031),
 ('warrior-king', 0.7232080698013306)]

result = model.most_similar(positive=['king', 'woman'], negative=['man'], topn=1)

print("King - Man + Woman =", result)

King - Man + Woman = [('queen', 0.778674840927124)]

result = model.most_similar(positive=['paris', 'italy'], negative=['france'], topn=1)

print("Paris - France + Italy =", result)

Paris - France + Italy = [('italia', 0.6415425539016724)]

vec_king = model['king']
vec_queen = model['queen']
vec_man = model['man']
vec_woman = model['woman']
```

```

cosine_similarity = np.dot(vec_king, vec_queen) / (np.linalg.norm(vec_king) *
np.linalg.norm(vec_queen))

print("Cosine Similarity between 'king' and 'queen':", cosine_similarity)

Cosine Similarity between 'king' and 'queen': 0.77042454

result = model.most_similar(positive=['uncle', 'woman'], negative=['man'], topn=1)

print("Uncle - Man + Woman =", result)

Uncle - Man + Woman = [('aunt', 0.8238280415534973)]

```

Exploring opposites: Good vs. Bad

```

print("Most similar words to 'good': " + " ".join(word for word, _ in model.most_similar('good')))
print("Most similar words to 'bad': " + " ".join(word for word, _ in model.most_similar('bad')))

```

Output

```

Most similar words to 'good': bad, goood, excellent, decent, .good, great, nice, not-so-good, good--and, good-enough
Most similar words to 'bad': good, good-bad, terrible, lousy, not-so-good, not-so-bad, horrible, awful, terrible, bad--

```

Viva Questions

1. What are word vectors and why are they useful in NLP?

Answer:

Word vectors are numerical representations of words in a high-dimensional space, where semantically similar words have similar vector representations. They're useful in NLP because they capture the contextual meaning of words, allowing algorithms to perform tasks like similarity comparison, analogy solving, and more.

2. What is the FastText model, and how is it different from Word2Vec or GloVe?

Answer:

FastText is a word embedding model developed by Facebook that represents words as a bag of character n-grams. Unlike Word2Vec or GloVe, which treat words as atomic units, FastText can generate vectors for out-of-vocabulary words by combining subword (character n-gram) information, making it more flexible and robust.

3. What does the vector size of 300 indicate in the FastText model used?

Answer:

The vector size of 300 means that each word (or subword) is represented as a 300-dimensional vector. This dimensionality defines the number of features or components used to represent each word in the embedding space.

4. What does `model.most_similar('king')` return?**Answer:**

It returns a list of words that are most semantically similar to "king", along with their similarity scores. These words are usually contextually related, such as "queen", "monarch", or "prince", depending on the training corpus.

5. What does the arithmetic operation `'king' - 'man' + 'woman'` represent, and what result did it give?**Answer:**

This operation represents a **word analogy**: "king is to man as X is to woman". The expected result is "queen", or a closely related term, demonstrating how vector arithmetic can model semantic relationships.

Program 2

Use dimensionality reduction (e.g., PCA or t-SNE) to visualize word embeddings for Q 1. Select 10 words from a specific domain (e.g., sports, technology) and visualize their embeddings. Analyze clusters and relationships. Generate contextually rich outputs using embeddings. Write a program to generate 5 semantically similar words for a given input.

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.decomposition import PCA

from sklearn.manifold import TSNE

from sklearn.cluster import KMeans

import gensim.downloader as api

model = api.load("word2vec-google-news-300")

words = ["computer", "AI", "algorithm", "network", "data", "cloud", "software", "hardware",
"internet", "machine"]

word_vectors = np.array([model[word] for word in words if word in model])

pca = PCA(n_components=10)
```

```
word_vectors_pca = pca.fit_transform(word_vectors)

tsne = TSNE(n_components=2, perplexity=30, learning_rate=200, n_iter=3000, random_state=42)
word_vectors_2D = tsne.fit_transform(word_vectors_pca)

num_clusters = 5 # Adjust number of clusters

kmeans = KMeans(n_clusters=num_clusters, random_state=42)

clusters = kmeans.fit_predict(word_vectors_2D)

colors = plt.cm.rainbow(np.linspace(0, 1, num_clusters))

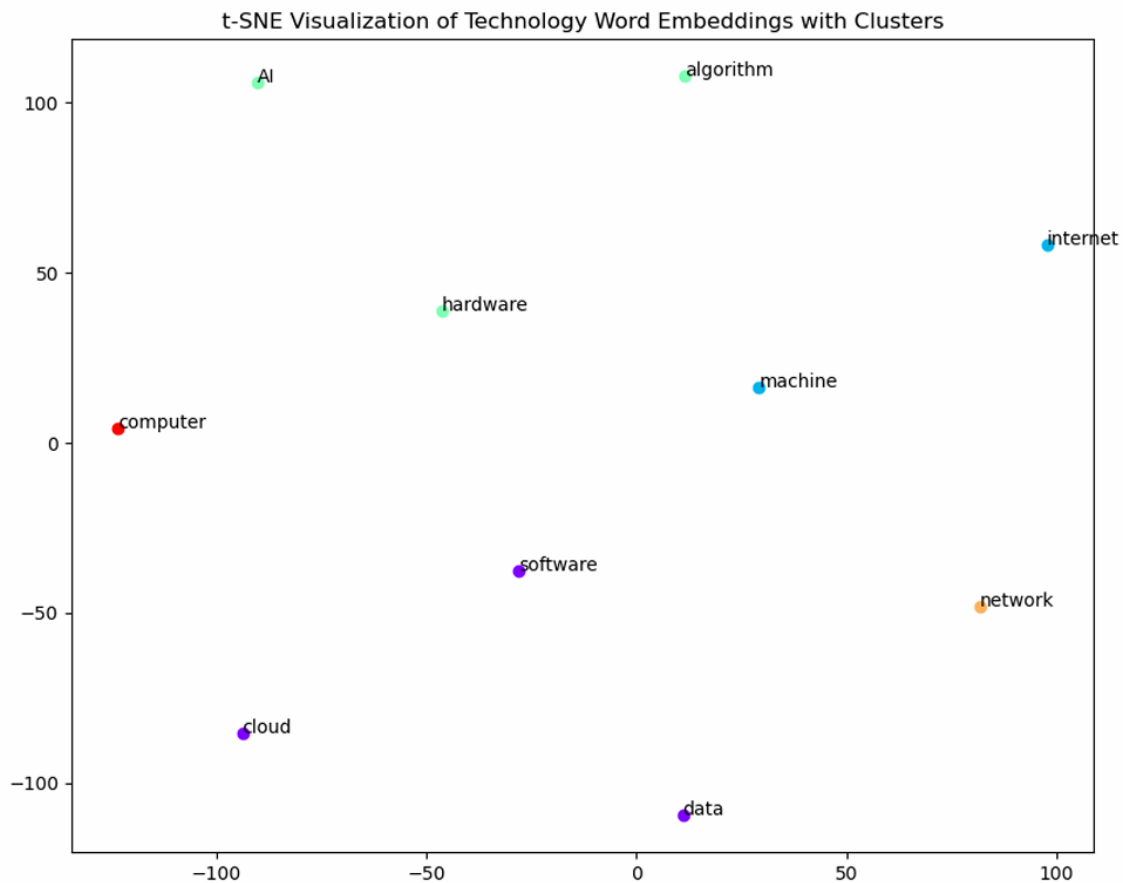
plt.figure(figsize=(10, 8))

for i, word in enumerate(words):
    if word in model:
        plt.scatter(word_vectors_2D[i, 0], word_vectors_2D[i, 1], color=colors[clusters[i]])
        plt.annotate(word, (word_vectors_2D[i, 0], word_vectors_2D[i, 1]))

plt.title("t-SNE Visualization of Technology Word Embeddings with Clusters")

plt.show()
```

Output



Next let's try an extra example with 55 words which shows the clustering where similar type of words group together (Not part of the question)

```
model = load('fasttext-wiki-news-subwords-300')
```

```
# List of tech-related words
```

```
words = [
```

```
    "AI", "algorithm", "automation", "backend", "bandwidth",
    "browser", "cache", "circuit", "cloud", "compiler", "computing",
    "cryptography", "cybersecurity", "database", "debugging",
    "docker", "encryption", "firewall",
    "framework", "frontend", "Git", "GitHub", "GPU", "hardware", "hacking",
    "HTML", "HTTPS", "IDE", "internet", "IoT", "JavaScript",
    "library", "malware", "microchip", "motherboard", "network",
    "NLP", "processor", "quantum", "RAM",
    "ransomware", "repository", "robotics", "router",
```

```

    "SaaS", "server", "software", "SSD", "statistics",
    "technology", "transistor", "virtualization",
    "voltage", "VPN", "web"
]

# Filter only the words that exist in the model
valid_words = [word for word in words if word in model]
word_vectors = np.array([model[word] for word in valid_words])

# Apply PCA to reduce dimensions before t-SNE
pca = PCA(n_components=30)
word_vectors_pca = pca.fit_transform(word_vectors)

# Apply t-SNE for 2D visualization
tsne = TSNE(n_components=2, perplexity=30, learning_rate=200, n_iter=3000, random_state=42)
word_vectors_2D = tsne.fit_transform(word_vectors_pca)

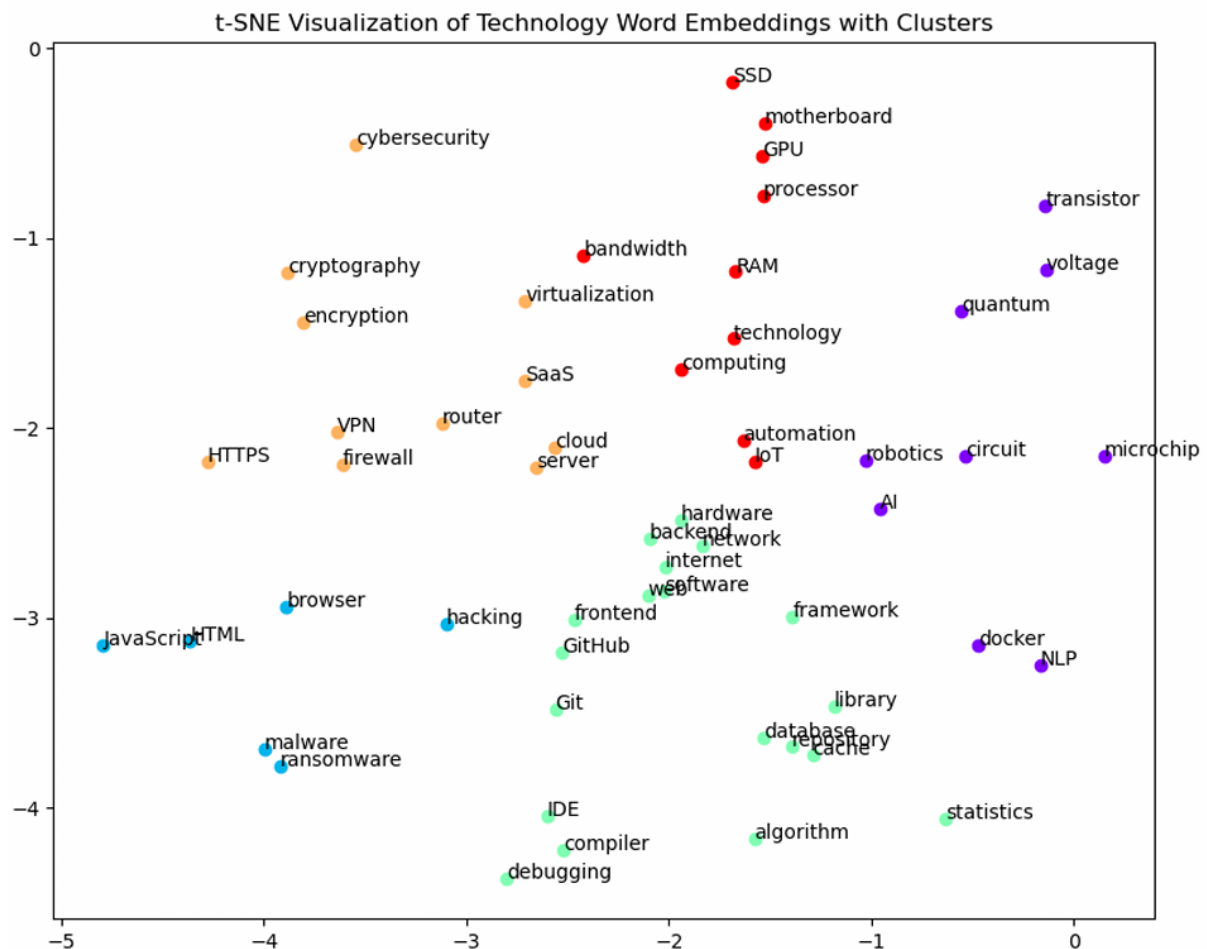
# Clustering with KMeans
num_clusters = 5
kmeans = KMeans(n_clusters=num_clusters, random_state=42)
clusters = kmeans.fit_predict(word_vectors_2D)

# Plotting the t-SNE result with clusters
colors = plt.cm.rainbow(np.linspace(0, 1, num_clusters))
plt.figure(figsize=(12, 10))

for i, word in enumerate(valid_words):
    plt.scatter(word_vectors_2D[i, 0], word_vectors_2D[i, 1], color=colors[clusters[i]], label=f"Cluster {clusters[i]}")
    if word == valid_words[clusters.tolist().index(clusters[i])] else ""
    plt.annotate(word, (word_vectors_2D[i, 0], word_vectors_2D[i, 1]))

```

```
plt.title("t-SNE Visualization of Technology Word Embeddings with Clusters")
plt.axis('off')
plt.legend()
plt.tight_layout()
plt.show()
```



Viva Questions

1. What is the role of PCA in this code?

Answer:

PCA (Principal Component Analysis) reduces the high-dimensional word vectors (e.g., 300 dimensions) to a lower-dimensional space (like 50) to remove noise and speed up computation, especially before applying t-SNE.

2. Why do we use PCA before applying t-SNE?

Answer:

t-SNE is computationally expensive and sensitive to noise. Applying PCA before t-SNE helps in reducing dimensions while preserving variance, which makes t-SNE faster and results more stable.

3. What is t-SNE, and how does it help in visualizing high-dimensional data?

Answer:

t-SNE (t-distributed Stochastic Neighbor Embedding) is a dimensionality reduction technique that converts high-dimensional data into 2D or 3D for visualization. It preserves local structure, making similar points appear close in the plot.

4. What do perplexity, learning_rate, and n_iter mean in t-SNE?

Answer:

- **Perplexity** balances attention between local and global aspects of data.
 - **Learning rate** controls how fast t-SNE converges.
 - **n_iter** is the number of iterations to run optimization (more iterations can lead to better results).
-

5. What would happen if you increase or decrease the number of PCA components?

Answer:

- **Increasing** PCA components might retain more information but slow down t-SNE.
 - **Decreasing** components too much might lose important data structure. An optimal balance is necessary (commonly 30–50 components).
-

Clustering and KMeans

6. What is KMeans clustering? How does it work?

Answer:

KMeans is an unsupervised algorithm that partitions data into **k clusters** based on feature

similarity. It assigns each point to the nearest cluster center and iteratively updates the centers until convergence.

7. Why did we choose 5 clusters in this code? Could we have chosen a different number?

Answer:

The number 5 was likely chosen based on prior knowledge or for demonstration. Yes, we could choose a different number depending on the dataset, and techniques like the **elbow method** can help decide the optimal k .

8. How is the KMeans algorithm using the 2D vectors generated by t-SNE?

Answer:

Once t-SNE reduces the vectors to 2D, KMeans uses these coordinates as input to group similar word points into clusters based on their spatial proximity.

9. What are some limitations of KMeans clustering in this kind of task?

Answer:

- It assumes spherical clusters and equal cluster size.
 - It's sensitive to initialization and outliers.
 - Doesn't perform well if the data structure is non-convex (which is common in word embeddings).
-

Matplotlib and Visualization

10. How are clusters visually distinguished in the scatter plot?

Answer:

Each cluster is represented with a different **color** and **label annotations**. This makes it easier to identify how words are grouped and how similar words lie near each other.

Program 3 :

Train a custom Word2Vec model on a small dataset. Train embeddings on a domain specific corpus (e.g., legal, medical) and analyze how embeddings capture domain-specific semantics.

```
import wikipediaapi

from gensim.models import Word2Vec
from gensim.utils import simple_preprocess
import random

# Initialize Wikipedia API
wiki_wiki = wikipediaapi.Wikipedia(user_agent='your-user-agent', language='en')

# List of medical topics to build the corpus
medical_topics = ["Diabetes", "Alzheimer's_disease", "Hypertension", "Cardiology", "Oncology"]

# Extract and build the medical corpus
medical_corpus = []

for topic in medical_topics:
    page = wiki_wiki.page(topic)
    if page.exists():
        sentences = page.text.split(". ")
        medical_corpus.extend(sentences)

# Randomly sample up to 500 sentences
medical_corpus = random.sample(medical_corpus, min(500, len(medical_corpus)))
```

```
# Tokenize the sentences
```

```
tokenized_corpus = [simple_preprocess(sentence) for sentence in medical_corpus if sentence.strip() != ""]
```

```
# Train Word2Vec model
```

```
model = Word2Vec(  
    sentences=tokenized_corpus,  
    vector_size=100,  
    window=5,  
    min_count=2, # Changed from 5 to 2 for small data, adjust as needed  
    workers=4,  
    sg=1 # Use skip-gram  
)
```

```
# Save and load the model
```

```
model.save("medical_word2vec_wiki.model")  
loaded_model = Word2Vec.load("medical_word2vec_wiki.model")
```

```
# Test similar words (ensure 'diabetes' exists in the vocab)
```

```
if 'diabetes' in loaded_model.wv:
```

```
    print("Words similar to 'diabetes':")
```

```
    for word, similarity in loaded_model.wv.most_similar("diabetes"):
```

```
        print(f"{word} : {similarity:.4f}")
```

```
else:
```

```
    print("'diabetes' not found in the vocabulary.")
```

Output

```
Words similar to 'diabetes': [('well', 0.997886598110199), ('autoimmune', 0.9977420568466187), ('type', 0.9977236986160278), ('people', 0.9975738525390625), ('association', 0.9975684285163879), ('complications', 0.9975566267967224), ('higher', 0.9975486993789673), ('is', 0.997524082660675), ('efficacy', 0.9975225925445557), ('common', 0.9975058436393738)]
```

Viva Questions:

1. What are word vectors and why are they important in NLP?

Explanation:

Word vectors are numerical representations of words in a high-dimensional space where similar words have similar vectors. They help algorithms understand and compare word meanings, enabling tasks like similarity detection, sentiment analysis, and machine translation.

2. What is Word2Vec, and how does it work?

Explanation:

Word2Vec is a model that learns word embeddings using two approaches:

- **CBOW (Continuous Bag of Words):** Predicts a word based on its context.
 - **Skip-Gram:** Predicts context words from a single word. It captures semantic relationships based on word co-occurrence.
-

3. What is FastText and how is it different from Word2Vec?

Explanation:

FastText (by Facebook) improves on Word2Vec by representing words as a sum of **character n-grams**. This allows it to handle **out-of-vocabulary words** better and understand subword structures like prefixes and suffixes.

4. Why do we use Wikipedia API and how do we build a corpus from it?

Explanation:

Wikipedia API allows access to article content directly from Wikipedia. We can extract page text and clean it to build a custom corpus for training models. It's a quick way to gather domain-specific data (like medical or tech topics).

5. What does `simple_preprocess()` do in text preprocessing?

Explanation:

`simple_preprocess()` tokenizes text, removes punctuation, converts it to lowercase, and filters out bad characters — preparing it for model training.

6. What is the purpose of PCA and why is it used before t-SNE?

Explanation:

PCA (Principal Component Analysis) reduces dimensionality while preserving variance. It's used before **t-SNE** to:

- Speed up computation
 - Remove noise
 - Improve t-SNE results stability
-

7. What is t-SNE and how does it help in visualization?

Explanation:

t-SNE reduces high-dimensional vectors into 2D/3D while preserving local relationships. It's perfect for **visualizing clusters or word similarity** in a way humans can understand.

8. What is KMeans clustering and how is it used here?

Explanation:

KMeans partitions data into k clusters based on similarity. After reducing vectors to 2D using t-SNE, KMeans groups similar words. It helps find **semantic word groups** (e.g., fruits, countries, diseases).

9. How does `most_similar()` work in Word2Vec or FastText?

Explanation:

It returns words that are **most similar** to a given word based on **cosine similarity**. For example, `model.most_similar("king")` might return ["queen", "prince", "monarch"] with similarity scores.

10. What is the meaning of vector arithmetic like `'king' - 'man' + 'woman'`?

Explanation:

This is a **word analogy** using vector math. It means:

"King is to man as ___ is to woman".

The result is often **"queen"**. This shows how word embeddings capture semantic relationships.

Program 4

Use word embeddings to improve prompts for Generative AI model. Retrieve similar words using word embeddings. Use similar words to enrich a GenAI prompt. Use the AI model to generate responses for the original and enriched prompts. Compare the outputs in terms of detail and relevance.

```
import gensim.downloader

import google.generativeai as genai

# Configure Gemini API
genai.configure(api_key="your_api_key")

# Load FastText embeddings
embeddings = gensim.downloader.load("fasttext-wiki-news-subwords-300")

# Function to find similar words
def find_similar_words(word, n=3):
    try:
        similar = [(w, s) for w, s in embeddings.most_similar(word.lower(), topn=10) if s > 0.6]
        return [w for w, _ in similar[:n]]
    except KeyError:
        return []

# Function to enrich the prompt with similar words
def enrich_prompt(prompt):
    words = prompt.split()
    enriched = []
    enriched_terms = set()

    for word in words:
```

```

clean_word = word.lower().strip('.,!?:;()')
if len(clean_word) > 3 and clean_word not in enriched_terms:
    similar = find_similar_words(clean_word)
    if similar:
        enriched.append(f"{word} (related concepts: {' '.join(similar)})")
        enriched_terms.add(clean_word)
    else:
        enriched.append(word)
else:
    enriched.append(word)

return " ".join(enriched)

```

Function to get Gemini response

```

def get_gemini_response(prompt):
    model = genai.GenerativeModel('gemini-2.0-flash-lite')
    response = model.generate_content(prompt)
    return response.text

```

Original and enriched prompts

```

original_prompt = "Write about AI in healthcare."
print(f"\nOriginal prompt: {original_prompt}")

```

```

enriched_prompt = enrich_prompt(original_prompt)
print(f"Enriched prompt: {enriched_prompt}")

```

Get responses

```

original_response = get_gemini_response(original_prompt)
enriched_response = get_gemini_response(enriched_prompt)

```



```
# Display responses
```

```
print("\n--- Original Prompt Response ---")
```

```
print(original_response[:500] + "..." if len(original_response) > 500 else original_response)
```

```
print("\n--- Enriched Prompt Response ---")
```

```
print(enriched_response[:500] + "..." if len(enriched_response) > 500 else
enriched_response)
```

```
# Comparison
```

```
print("\nComparison:")
```

```
print(f"Original response length: {len(original_response.split())} words")
```

```
print(f"Enriched response length: {len(enriched_response.split())} words")
```

```
print(f"Original unique words: {len(set(original_response.lower().split()))}")
```

```
print(f"Enriched unique words: {len(set(enriched_response.lower().split()))}")
```

Output

```
Original prompt: Write about AI in healthcare.
Enriched prompt: Write (related concepts: read, writing, publish) about (related concepts: abouth, aabout, aboout) AI in healthcare. (related c
oncepts: health-care, non-healthcare, heathcare)

--- Original Prompt Response ---
## The AI Revolution in Healthcare: Reshaping the Future of Medicine

Artificial intelligence (AI) is rapidly transforming healthcare, promising to revolutionize everything from diagnosis and treatment to patient
management and drug discovery. This technology's ability to analyze vast datasets, identify patterns, and automate tasks is unlocking unprecede
nted opportunities to improve patient outcomes, reduce costs, and enhance the efficiency of healthcare systems.

**Here's a breakdown of how AI ...

--- Enriched Prompt Response ---
## AI in Healthcare: Reading, Writing & Publishing the Future

The intersection of Artificial Intelligence (AI) and healthcare represents a transformative frontier, with profound implications for how we **r
ead** medical data, **write** medical solutions, and **publish** the future of patient care. This journey of exploration delves into the curren
t and potential impacts of AI across various facets of the **healthcare** landscape, differentiating it from **non-healthcare** applications.

**Readin...

Comparison:
Original response length: 839 words
Enriched response length: 515 words
Original unique words: 443
Enriched unique words: 319
```

Viva Questions:

1. What is the main purpose of this code?

Explanation:

The code takes a user prompt (e.g., *“Write about AI in healthcare”*), enriches it by adding **related terms** using **FastText word embeddings**, and then compares the responses generated by **Gemini AI** for both the original and enriched prompts.

2. What are FastText embeddings, and how are they used here?

Explanation:

FastText provides **word vectors** that capture semantic meaning, including subword information. In this code, it finds **semantically related words** to enrich the original prompt for better AI understanding and response.

3. How does the `find_similar_words()` function work?

Explanation:

This function uses FastText’s `most_similar()` method to find words similar to the input word. It filters those with similarity scores **above 0.6** and returns the **top N** related terms.

4. What is the role of the `enrich_prompt()` function?

Explanation:

It enhances the user’s prompt by appending similar/related terms next to key words (e.g., *“AI (related concepts: algorithm, machine, intelligence)”*), helping the AI model generate more detailed and relevant content.

5. Why is `strip('.,!?:;()')` used in `enrich_prompt()`?

Explanation:

It removes punctuation around each word to ensure only clean words are sent to the embedding model. This avoids **KeyErrors** and improves similarity matching.

6. What does `genai.GenerativeModel('gemini-2.0-flash-lite')` do?

Explanation:

It initializes the **Gemini AI model** (from Google) using a **lightweight version** for faster generation. It’s used to generate content from both the original and enriched prompts.

7. What is the purpose of comparing word counts and uniqueness at the end?

Explanation:

The comparison helps evaluate how **enriching the prompt** affects the **length, diversity, and richness** of the AI-generated output, giving insights into response quality.

8. What happens if a word is not found in FastText embeddings?

Explanation:

The `find_similar_words()` function catches a `KeyError` and returns an empty list. This ensures the program continues smoothly even if some words are out-of-vocabulary.

9. Why are only words with length > 3 used for enrichment?

Explanation:

Short words (like *"a"*, *"is"*, *"in"*) are typically stopwords or less meaningful in context. Filtering them out helps focus on **content-heavy** keywords for meaningful enrichment.

10. How does enriching a prompt help in prompt engineering?

Explanation:

Enriching prompts with **related concepts** gives the AI **more context**, which often results in **richer, more accurate, and more creative responses** — a key technique in prompt engineering.

Program 5

Use word embeddings to create meaningful sentences for creative tasks. Retrieve similar words for a seed word. Create a sentence or story using these words as a starting point. Write a program that: Takes a seed word. Generates similar words. Constructs a short paragraph using these words.

```
import gensim.downloader as api
```

```
import random
```

```

import numpy as np

from transformers import pipeline

import google.generativeai as genai

# Configure the Gemini API
genai.configure(api_key="your_api_key")

# Cosine similarity function
def cosine_similarity(vec1, vec2):
    return np.dot(vec1, vec2) / (np.linalg.norm(vec1) * np.linalg.norm(vec2))

# Get similar words using word embeddings
def get_similar_words(word, model, top_n=5):
    try:
        word_vec = model[word]
        all_words = model.index_to_key
        similarities = [
            (w, cosine_similarity(word_vec, model[w]))
            for w in all_words if w.isalpha()
        ]
        similarities.sort(key=lambda x: x[1], reverse=True)
        return [w[0] for w in similarities[:top_n]]
    except KeyError:
        return []

# Generate a paragraph using Gemini based on similar words
def generate_paragraph(seed_word, model):
    similar_words = get_similar_words(seed_word, model)

```

```

if not similar_words:

    return f"Couldn't find similar words for '{seed_word}'. Try another word."

print("Similar words are:\n", similar_words, "\n")

prompt = f"Write a short, creative paragraph using these words: {'',
'.join(similar_words)}.\n"

gemini_model = genai.GenerativeModel("gemini-2.0-flash-lite")

response = gemini_model.generate_content(prompt)

return response.text if response.text else "Failed to generate text."

# Load pre-trained word2vec model

embedding_model = api.load("word2vec-google-news-300")

# Generate paragraph using a seed word

seed_word = "ship"

paragraph = generate_paragraph(seed_word, embedding_model)

print(paragraph)

```

Output

```

Similar words are :
['ship', 'ships', 'vessel', 'warship', 'freighter']

```

```

The crimson sunset bled across the horizon as the lone ship cut through the waves, its silhouette a stark contrast to the fiery sky. Below, a fleet of smaller ships bobbed gently, their hulls reflecting the dying light. This was no casual cruise, however. The colossal freighter, a behemoth of cargo and commerce, was flanked by the ominous presence of a powerful warship, its gun turrets a silent promise of protection. Each vessel, a distinct player in this watery ballet, held its own secrets, navigating a world teetering on the edge of peace and conflict.

```

Viva Questions:

1. What is the purpose of this code?

Explanation:

The code takes a **seed word**, finds similar words using **Word2Vec embeddings**, then generates a **creative paragraph** using those similar words by prompting the **Gemini AI model**.

2. What does the `word2vec-google-news-300` model contain?

Explanation:

It's a **pre-trained Word2Vec model** trained on **Google News**, containing 3 million English words and phrases, each represented as a **300-dimensional vector**. It helps capture semantic similarity between words.

3. How does the `cosine_similarity()` function work?

Explanation:

It calculates how **similar two word vectors are** based on the **cosine of the angle** between them. A score closer to **1** means high similarity; closer to **0** means dissimilar.

4. What does the `get_similar_words()` function do?

Explanation:

It finds the **top N words** most similar to the given word using **cosine similarity** between their embeddings. It filters for alphabetic words and sorts by similarity score.

5. What is the role of the `generate_paragraph()` function?

Explanation:

This function:

1. Retrieves similar words for a seed word
 2. Creates a prompt using those words
 3. Sends the prompt to **Gemini AI** to generate a creative paragraph
-

6. Why is `word.isalpha()` used when finding similar words?

Explanation:

To **exclude non-alphabetic entries** (like numbers, symbols, phrases) from the model's vocabulary. This keeps the generated prompt cleaner and more meaningful.

7. Why use Gemini AI after finding similar words?

Explanation:

While Word2Vec gives related terms, **Gemini** (a generative model) can **create coherent text** using those words. This showcases the integration of **static embeddings** with **dynamic content generation**.

8. What happens if the seed word is not in the vocabulary?**Explanation:**

The code handles this using a **try-except block**. If the seed word is missing from the model, it returns a friendly error message instead of crashing.

9. How is this different from traditional prompt writing?**Explanation:**

Here, the prompt is **automatically enriched** using **semantically similar words**, making it more **creative and diverse**, which enhances the quality of the response generated by Gemini.

10. What are the practical applications of this code?**Explanation:**

- **Creative writing assistance**
- **Content idea generation**
- **Prompt enrichment in AI apps**

Semantic exploration of concepts

Program 6:

Use a pre-trained Hugging Face model to analyze sentiment in text. Assume a real-world application, Load the sentiment analysis pipeline. Analyze the sentiment by giving sentences to input

```
from transformers import pipeline
```

```
# Load the sentiment analysis pipeline
```

```
sentiment_pipeline = pipeline(model="distilbert/distilbert-base-uncased-finetuned-sst-2-english")
```

```
# Function to analyze sentiment
def analyze_sentiment(text):
    result = sentiment_pipeline(text)
    return result

# Sample customer reviews
customer_reviews = [
    "The product quality is amazing! I'm very satisfied with my purchase.",
    "Terrible experience. The item arrived broken and customer service was unhelpful.",
    "It's okay, not the best but does the job.",
    "Absolutely love it! Highly recommended to everyone.",
    "Worst purchase ever. Do not waste your money on this."
]

# Analyze each review
for review in customer_reviews:
    sentiment = analyze_sentiment(review)
    print(f"Review: {review}\nSentiment: {sentiment}\n")
```

Output

Review: The product quality is amazing! I'm very satisfied with my purchase.
Sentiment: [{'label': 'POSITIVE', 'score': 0.9998836517333984}]

Review: Terrible experience. The item arrived broken and customer service was unhelpful.
Sentiment: [{'label': 'NEGATIVE', 'score': 0.9996428489685059}]

Review: It's okay, not the best but does the job.
Sentiment: [{'label': 'POSITIVE', 'score': 0.9998031258583069}]

Review: Absolutely love it! Highly recommended to everyone.
Sentiment: [{'label': 'POSITIVE', 'score': 0.9998792409896851}]

Review: Worst purchase ever. Do not waste your money on this.
Sentiment: [{'label': 'NEGATIVE', 'score': 0.9991238713264465}]

Viva Questions:

1. What is the purpose of this code?

Explanation:

The code analyzes the **sentiment** (positive/negative) of a list of customer reviews using a **pre-trained DistilBERT model** fine-tuned on the SST-2 sentiment classification dataset.

2. What is `pipeline()` in Hugging Face Transformers?

Explanation:

`pipeline()` is a high-level API that simplifies using models for tasks like **sentiment analysis, text generation, translation**, etc. It loads everything (tokenizer + model) and makes inference easy.

3. What does the model `distilbert/distilbert-base-uncased-finetuned-sst-2-english` do?

Explanation:

It's a **DistilBERT** model fine-tuned on the **Stanford Sentiment Treebank (SST-2)** dataset for binary classification: **positive** or **negative** sentiment.

4. What is DistilBERT and why is it used here?

Explanation:

DistilBERT is a lightweight, faster version of BERT. It maintains **95% accuracy** of BERT while being more efficient, making it ideal for quick NLP tasks like sentiment analysis.

5. How does `sentiment_pipeline(text)` work?

Explanation:

It tokenizes the input text, passes it through the pre-trained model, and returns the **sentiment label** (`POSITIVE` or `NEGATIVE`) along with a **confidence score**.

6. What kind of output does `analyze_sentiment()` return?

Explanation:

It returns a list with a dictionary, e.g.:

```
[{'label': 'POSITIVE', 'score': 0.9998}]
```

7. What types of input does this sentiment model handle best?

Explanation:

The model performs best on **short, opinionated English text**, such as **reviews, tweets, or user comments**. It's less effective on neutral or multi-opinion texts.

8. How can the model's performance be evaluated or improved?

Explanation:

- Use a **custom dataset** for fine-tuning if needed.
 - Evaluate with **precision, recall, F1-score** on a labeled test set.
 - Improve performance using **ensemble models** or domain-specific fine-tuning.
-

9. What is the limitation of binary sentiment analysis (like SST-2)?

Explanation:

It classifies only as **positive or negative**. It can't identify **neutral** sentiments or **emotions** like anger, sadness, or joy unless extended or replaced by a multi-class model.

10. How could this code be adapted for real-world applications?

Explanation:

It can be used to:

- Analyze **customer feedback** on e-commerce platforms
- Automate **review moderation**
- Monitor **brand sentiment** on social media
- Power **chatbots** to respond empathetically

Program 7:

Summarize long texts using a pre-trained summarization model using Hugging face model. Load the summarization pipeline. Take a passage as input and obtain the summarized text.

```
from transformers import pipeline
```

```
summarization_pipeline = pipeline ("summarization", model="facebook/bart-large-cnn")
```

```
def summarize_text(text, max_length=100, min_length=30):
```

```

summary = summarization_pipeline(text, max_length=max_length, min_length=min_length,
do_sample=False)

return summary[0]['summary_text']

long_text = ( "Artificial intelligence (AI) is transforming industries across the globe. Companies are
integrating AI-driven solutions to "

"enhance efficiency, automate repetitive tasks, and improve decision-making. In healthcare, AI is
being used for early disease "

"detection, personalized treatments, and robotic surgeries. Finance sectors are leveraging AI for
fraud detection, risk assessment, "

"and automated trading. Despite its benefits, AI also raises ethical concerns, including bias in
decision-making, job displacement, "

"and data privacy issues. Policymakers and researchers are actively working on frameworks to
ensure responsible AI development and deploy" )

summary = summarize_text(long_text)

print("Original Text:", long_text)

print("\nSummarized Text:", summary)

```

Output

Original Text: Artificial intelligence (AI) is transforming industries across the globe. Companies are integrating AI-driven solutions to enhance efficiency, automate repetitive tasks, and improve decision-making. In healthcare, AI is being used for early disease detection, personalized treatments, and robotic surgeries. Finance sectors are leveraging AI for fraud detection, risk assessment, and automated trading. Despite its benefits, AI also raises ethical concerns, including bias in decision-making, job displacement, and data privacy issues. Policymakers and researchers are actively working on frameworks to ensure responsible AI development and deployment.

Summarized Text: Artificial intelligence (AI) is transforming industries across the globe. In healthcare, AI is being used for early disease detection, personalized treatments, and robotic surgeries. Finance sectors are leveraging AI for fraud detection, risk assessment, and automated trading.

Viva Questions:

1. What is the main goal of this code?

Explanation:

The code takes a **long input paragraph** and generates a **shorter summary** using a **pre-trained BART model** (`facebook/bart-large-cnn`) for abstractive summarization.

2. What model is used for summarization and why?

Explanation:

The model used is `facebook/bart-large-cnn`.

BART is a sequence-to-sequence model trained for tasks like summarization, translation, and text generation. The **CNN** part indicates it's fine-tuned on **news articles**, making it ideal for summarizing factual content.

3. What is the `pipeline()` function used for in this code?**Explanation:**

`pipeline()` is a high-level API from Hugging Face that simplifies using models for common tasks like summarization, translation, or sentiment analysis. It loads both the tokenizer and model internally.

4. What do `max_length` and `min_length` mean in summarization?**Explanation:**

- `max_length`: The **maximum number of tokens** in the generated summary.
 - `min_length`: The **minimum number of tokens** in the summary.
- These control the size and detail level of the output summary.
-

5. What is `do_sample=False` and why is it used?**Explanation:**

Setting `do_sample=False` makes the model use **greedy decoding** instead of sampling randomly. This ensures **deterministic, more consistent** summaries — especially for factual text.

6. How does abstractive summarization differ from extractive summarization?**Explanation:**

- **Abstractive summarization** (like BART) **generates new sentences** based on the meaning of the input.
 - **Extractive summarization** selects and combines **existing sentences or phrases** from the input.
- Abstractive is more flexible but also more complex.
-

7. What happens if the input text is too short or too long?

Explanation:

- If too **short**, the summary might be unnecessary or too similar to the input.
 - If too **long**, it may exceed the model's token limit (e.g., 1024 tokens for BART), which could **truncate** the input or raise an error.
-

8. Can this summarization model be used for documents in other languages?

Explanation:

Not directly. `facebook/bart-large-cnn` is trained on **English**. For other languages, you'd need **multilingual models** like **mBART** or language-specific fine-tuned models.

9. What is the structure of the output returned by `summarization_pipeline`?

Explanation:

The output is a list of dictionaries. Example:

```
[{'summary_text': '...summary here...'}]
```

So, we access the summary with `summary[0]['summary_text']`.

10. What are practical applications of this summarization pipeline?

Explanation:

- Summarizing **news articles**, **research papers**, **meeting notes**
- Creating **short previews** for blogs or reports
- Helping with **document analysis** and **automated briefing systems**

Program 8:

Install `langchain`, `cohere` (for key), `langchain-community`. Get the api key(By logging into Cohere and obtaining the cohere key). Load a text document from your google drive . Create a prompt template to display the output in a particular manner.

```
from googleapiclient.discovery import build
from googleapiclient.http import MediaIoBaseDownload
from google.oauth2 import service_account
import io

# Define the scope for Google Drive API
SCOPES = ['https://www.googleapis.com/auth/drive']
SERVICE_ACCOUNT_FILE = "credentials.json"

# Authenticate using service account
creds = service_account.Credentials.from_service_account_file(
    SERVICE_ACCOUNT_FILE, scopes=SCOPES
)
service = build('drive', 'v3', credentials=creds)

# Function to download a file from Google Drive
def download_file(file_id):
    request = service.files().get_media(fileId=file_id)
    file_stream = io.BytesIO()
    downloader = MediaIoBaseDownload(file_stream, request)
    done = False
    while not done:
        status, done = downloader.next_chunk()
        print(f"Download {int(status.progress() * 100)}%.")
    return file_stream.getvalue().decode("utf-8")
```

```
# Replace this with your actual file ID from Google Drive
```

```
file_id = "file_id_from_google_drive"
```

```
text_content = download_file(file_id)
```

```
print("Loaded Text:\n", text_content[:500])
```

```
# LangChain & Cohere integration
```

```
from langchain.llms import Cohere
```

```
from langchain.prompts import PromptTemplate
```

```
from langchain.chains import LLMChain
```

```
# Initialize Cohere LLM with your API key
```

```
cohere_api_key = "your_api_key"
```

```
llm = Cohere(cohere_api_key=cohere_api_key)
```

```
# Define a prompt template for summarization
```

```
template = """
```

```
Summarize the following text in a concise and structured manner:
```

```
TEXT:
```

```
{text}
```


Summary:

"""

Prepare the prompt

```
prompt = PromptTemplate(template=template, input_variables=["text"])
```

Create and run the LangChain chain

```
chain = LLMChain(llm=llm, prompt=prompt)
```

```
summary = chain.run(text_content)
```

Output the generated summary

```
print("Generated Summary:\n", summary)
```

Output

Generated Summary:

Here's a brief summary of the text:

Artificial intelligence (AI) is rapidly invading many industries, offering improvements in efficiency, automation, and decision-making. AI benefits include early disease detection, personalized treatments, and robotic surgeries in healthcare; fraud detection, risk assessment, and automated trading in finance. Nonetheless, the adoption of AI also raises ethical concerns over bias, job displacement, and data privacy. Active efforts are underway to develop frameworks for the responsible implementation of AI.

Let me know if you'd like me to elaborate on any of the points or provide additional information.

Viva Questions:

1. What is the purpose of this code?

Explanation:

This code downloads a text file from **Google Drive** using the **Drive API**, reads its content, and then sends it to a **Cohere LLM** through **LangChain** to generate a structured summary.

2. What is the role of `service_account.Credentials` and `credentials.json`?

Explanation:

`credentials.json` contains a **service account key** for authenticating with Google APIs. `service_account.Credentials` uses this to grant access to the **Google Drive API** securely.

3. What does the `download_file(file_id)` function do?**Explanation:**

This function uses the **Google Drive API** to download a file by its ID, reads it into memory using a `BytesIO` stream, and decodes it as a UTF-8 string for further processing.

4. Why do we use `MediaIoBaseDownload`?**Explanation:**

`MediaIoBaseDownload` is a Google API utility that handles **chunked file downloading** from Drive. It's useful for downloading large files reliably.

5. What is Cohere and how is it used in this code?**Explanation:**

Cohere is an AI platform offering powerful LLMs. Here, it's used (via **LangChain's Cohere wrapper**) to generate summaries from the input text content retrieved from Google Drive.

6. What does `PromptTemplate` do in LangChain?**Explanation:**

`PromptTemplate` defines a **custom prompt format** that the LLM should follow. It allows inserting variables like the downloaded `text` into a predefined prompt structure for consistent output.

7. What is `LLMChain` and how does it work in this context?**Explanation:**

`LLMChain` is a **LangChain object** that combines a prompt and an LLM to form a processing pipeline. In this code, it takes the prompt + text and calls Cohere to generate the summary.

8. Why is `.run(text_content)` used in `chain.run()`?

Explanation:

`.run()` sends the actual text (downloaded from Drive) into the **LLM pipeline**, using the defined prompt structure, and receives the summarized output from the Cohere model.

9. What are the benefits of using LangChain in this setup?

Explanation:

LangChain simplifies **prompt chaining**, **model abstraction**, and integration with tools like Cohere, OpenAI, and even external tools like file systems or APIs. It makes **LLM workflows modular and scalable**.

10. What are some real-world applications of this code?

Explanation:

- Summarizing long documents stored in Google Drive
- Automating report generation from shared files
- Building internal AI tools for enterprise file summarization
- Enhancing productivity in research or legal review

Program 9:

Take the Institution name as input. Use Pydantic to define the schema for the desired output and create a custom output parser. Invoke the Chain and Fetch Results. Extract the below Institution related details from Wikipedia: The founder of the Institution. When it was founded. The current branches in the institution . How many employees are working in it. A brief 4-line summary of the institution.

```
import wikipedia
```

```
from pydantic import BaseModel
```

```
from typing import Optional
```

```
# Define the data model using Pydantic
```

```
class InstitutionInfo(BaseModel):
```

```
    name: str
```

```
    founder: Optional[str] = None
```

```
    founded_year: Optional[str] = None
```

```
    branches: Optional[str] = None
```

```
    employees: Optional[str] = None
```

```
    summary: Optional[str] = None
```

```
# Parser class to extract information from Wikipedia
```

```
class InstitutionParser:
```

```
    @staticmethod
```

```
    def parse_wikipedia_summary(institution_name: str) ->  
Optional[InstitutionInfo]:
```

```
    try:
```

```
        summary = wikipedia.summary(institution_name, sentences=5)
```

```
        page = wikipedia.page(institution_name)
```

```
        content = page.content.lower()
```

```
        founder = InstitutionParser.extract_info(content, ["founder", "founded  
by"])
```

```
        founded_year = InstitutionParser.extract_info(content, ["founded",  
"established"])
```

```
        branches = InstitutionParser.extract_info(content, ["branches",  
"campuses", "locations"])
```

```
        employees = InstitutionParser.extract_info(content, ["employees",  
"staff", "faculty"])
```

```

    return InstitutionInfo(
        name=institution_name,
        founder=founder,
        founded_year=founded_year,
        branches=branches,
        employees=employees,
        summary=summary
    )
except Exception as e:
    print(f"Error fetching details for {institution_name}: {e}")
    return None

```

@staticmethod

```

def extract_info(content: str, keywords: list) -> Optional[str]:
    for keyword in keywords:
        index = content.find(keyword)
        if index != -1:
            snippet = content[index:index + 100]
            return snippet.split(".")[0]
    return None

```

User input and output display

```
institution_name = input("Enter the Institution name: ")
```

```
institution_details =
```

```
InstitutionParser.parse_wikipedia_summary(institution_name)
```

```
if institution_details:
```

```

print(f"\nInstitution Name: {institution_details.name}")

print(f"Founder: {institution_details.founder if institution_details.founder
else 'Not Available'}")

print(f"Founded Year: {institution_details.founded_year if
institution_details.founded_year else 'Not Available'}")

print(f"Branches: {institution_details.branches if institution_details.branches
else 'Not Available'}")

print(f"Employees: {institution_details.employees if
institution_details.employees else 'Not Available'}")

print(f"\nSummary:\n{institution_details.summary if
institution_details.summary else 'Not Available'}\n")

```

Output

```

Institution Name: AMC Engineering College
Founder: Not Available
Founded Year: established in the year 1999 under the paramahansa foundation trust
Branches: Not Available
Employees: Not Available

```

Summary:

AMC Group of Institutions are educational institutions in Bangalore, India. The group primarily operate higher education institutions like AMC College of Hotel Management, AMC Engineering College, Advanced Business Management College, AMC PU College, its spin-off branch: AMC College (Jayanagar) and City College of Engineering. The institutions are situated in a combined 55-acre (220,000 m²) campus' with head office and main branch in Bannerghatta Road, Bangalore. It is accredited as a group B- institute by NBA-AICTE in 2017 and is approved for operation by the All India Council for Technical Education (AICTE), New Delhi. Colloquially, AMC College refers to the engineering college, which is the primary source of income.

Viva Questions:

1. What is the purpose of this code?

Explanation:

This code takes the name of an institution as input, fetches its summary from **Wikipedia**, and attempts to extract specific information (like founder, founded year, branches, and employees) in a **structured format** using a `Pydantic` model.

2. What is `Pydantic` and why is it used here?

Explanation:

Pydantic is a data validation library for Python based on type hints. Here, it's used to define

a **clean, validated data model** (`InstitutionInfo`) for holding institutional attributes like name, founder, and summary.

3. What is the role of the `InstitutionInfo` class?

Explanation:

`InstitutionInfo` is a Pydantic `BaseModel` that defines the structure of the data being extracted. It ensures that the extracted info is **typed, consistent**, and easy to work with.

4. How does the code fetch information from Wikipedia?

Explanation:

It uses the `wikipedia` Python package. The method `wikipedia.summary()` gets a short text summary, while `wikipedia.page().content` fetches the full article content for keyword-based parsing.

5. What does the `extract_info()` method do?

Explanation:

This method searches the Wikipedia article's content for specific **keywords** (like "founder", "branches") and returns a snippet of text around the matched term — ideally containing the desired information.

6. How does the code handle missing or unavailable data?

Explanation:

If a keyword is not found, `extract_info()` returns `None`. When printing results, the code checks if each field is available; if not, it displays "Not Available" instead of throwing errors.

7. Why is `.lower()` used on the Wikipedia page content?

Explanation:

`.lower()` ensures **case-insensitive keyword matching**. This increases the chance of finding relevant information, regardless of how the keyword appears in the article.

8. What kind of limitations does this approach have?

Explanation:

- It uses **basic string search**, which may miss info or return inaccurate snippets.
 - It doesn't use NLP or structured parsing, so the extracted data might not be clean or exact.
 - Reliant on how Wikipedia articles are written.
-

9. How can this script be improved for more accurate extraction?

Explanation:

- Use **Named Entity Recognition (NER)** to detect names and dates.
 - Use **regex** to extract numbers or patterns.
 - Use **structured sources** like DBpedia or Wikidata for reliable info.
 - Add **natural language processing** for context-aware parsing.
-

10. What happens if the institution page doesn't exist on Wikipedia?

Explanation:

The `try-except` block catches any exceptions (like `PageError` or `DisambiguationError`) and prints an error message, ensuring the program doesn't crash and instead returns `None`.

Program 10:

Build a chatbot for the Indian Penal Code. We'll start by downloading the official Indian Penal Code document, and then we'll create a chatbot that can interact with it. Users will be able to ask questions about the Indian Penal Code and have a conversation with it.

```
import PyPDF2

from sentence_transformers import SentenceTransformer

import numpy as np

from sklearn.metrics.pairwise import cosine_similarity


# Extract text from a PDF file
def extract_text_from_pdf(pdf_path):
    text = ""
    with open(pdf_path, "rb") as file:
```



```
reader = PyPDF2.PdfReader(file)
for page in reader.pages:
    text += page.extract_text() + "\n"
return text
```

Split text into chunks

```
def split_text(text, chunk_size=500):
    sentences = text.split(". ")
    chunks = []
    chunk = ""
    for sentence in sentences:
        if len(chunk) + len(sentence) < chunk_size:
            chunk += sentence + ". "
        else:
            chunks.append(chunk.strip())
            chunk = sentence + ". "
    if chunk:
        chunks.append(chunk.strip())
    return chunks
```

Interactive IPC Chatbot

```
def ipc_chatbot(chunks, model):
    chunk_embeddings = model.encode(chunks)
    print("\nIPC Chatbot: Ask me about the Indian Penal Code! Type 'exit' to quit.")

    while True:
```

```

query = input("You: ")
if query.lower() == "exit":
    print("Exiting chatbot...")
    break

query_embedding = model.encode([query])
similarities = cosine_similarity(query_embedding, chunk_embeddings)
best_match_idx = np.argmax(similarities)

print("IPC Bot:", chunks[best_match_idx])

# Load PDF, process, and start chatbot
pdf_path = "indian penal code.pdf" # Make sure this file exists
extracted_text = extract_text_from_pdf(pdf_path)
chunks = split_text(extracted_text)
model = SentenceTransformer("all-MiniLM-L6-v2")
ipc_chatbot(chunks, model)

```

Output

```

IPC Chatbot: Ask me about the Indian Penal Code! Type 'exit' to quit.
IPC Bot: Whoever commits theft–
(a) in any building, tent or vessel used as a human dwelling or used for the
custody of property; or
(b) of any means of transport used for the transport of goods or passengers; or
(c) of any article or goods from any means of transport used for the transport of
goods or passengers; or
(d) of idol or icon in any place of worship; or
(e) of any property of the Government or of a local authority,
shall be punished with imprisonment of either description for a term which may extend to
seven years and shall also be liable to fine.
304.
Exiting chatbot...

```

Viva Questions:

1. What is the purpose of this code?

Explanation:

This code builds an **interactive chatbot** that answers questions related to the **Indian Penal Code** by retrieving the most relevant section from a **PDF file** using **semantic similarity**.

2. How does the code extract data from the PDF file?

Explanation:

It uses the `PyPDF2` library to read and extract text from each page of the PDF file. The function `extract_text_from_pdf()` opens the file in binary mode, iterates through pages, and combines the extracted text.

3. Why is the text split into chunks and how is it done?

Explanation:

The PDF content is split into manageable **chunks** to allow meaningful semantic comparisons. The `split_text()` function breaks the content into **sentence-based segments** of approximately 500 characters.

4. What is the role of the `SentenceTransformer` model?

Explanation:

`SentenceTransformer` generates **dense vector representations (embeddings)** of sentences or paragraphs. These embeddings are used to measure **semantic similarity** between the user's question and each chunk.

5. Which pre-trained model is used here and why?

Explanation:

The code uses "`all-MiniLM-L6-v2`", a compact but powerful **sentence transformer** that balances **speed and accuracy**. It's suitable for semantic search and real-time applications like chatbots.

6. How does the chatbot find the best response to a user's query?

Explanation:

The chatbot:

- Encodes the user's input using the sentence transformer
 - Computes **cosine similarity** between the query embedding and all chunk embeddings
 - Returns the chunk with the **highest similarity score** as the most relevant answer
-

7. What does `cosine_similarity()` measure in this context?

Explanation:

It measures the **semantic closeness** between the query and each chunk based on their vector representations. A score closer to **1.0** indicates higher relevance.

8. What are the limitations of this chatbot approach?

Explanation:

- It retrieves exact text chunks instead of generating dynamic answers
 - Accuracy depends on how well the text is chunked
 - Cannot handle very specific or deeply contextual questions without additional logic
-

9. What is the importance of chunk size in text splitting?

Explanation:

Chunk size affects the **granularity** of responses:

- Too small: Important context might be lost
 - Too large: Matching might become vague or irrelevant
- A balanced chunk size helps in better matching and user satisfaction.
-

10. How can this chatbot be improved?

Explanation:

- Use **question answering models** like BERT QA for more precise answers
- Highlight matched keywords in the output
- Add **memory** to track previous questions
- Build a **GUI or web app** using Streamlit or Flask for better UX

